

บทที่ 14

แนวความคิดการเขียนโปรแกรมเชิงวัตถุขั้นสูง

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรณโรจน์

หัวข้อ

- Dependency Injection
- คลาสและอินเตอร์เฟสแบบ Sealed
- คลาสภายใน (Inner class หรือ Nested class)
- Model-View-Controller (MVC) Design Pattern

หัวข้อ

- Dependency Injection
- คลาสและอินเตอร์เฟสแบบ Sealed
- คลาสภายใน (Inner class หรือ Nested class)
- Model-View-Controller (MVC) Design Pattern

Dependency Injection



Dependency Injection เป็นเทคนิคหนึ่งในการลดการพึ่งพาโดยตรงระหว่างคลาสหนึ่งกับคลาสอื่น ๆ ในโปรแกรม ซึ่งช่วยให้โปรแกรมมีการออกแบบที่ยืดหยุ่นมากขึ้น, ง่ายต่อการทดสอบ, และมีการแยกส่วนที่ชัดเจนระหว่าง components การทำ Dependency Injection สามารถ inject เกิดขึ้นตอนสร้างอินสแตนซ์ของคลาส แทนที่จะให้คลาสนั้นสร้าง dependencies ของมันเอง มันสามารถทำได้หลายวิธี เช่น ผ่านเมธอด constructor, setter หรือเมธอดอื่น ๆ

```
public interface Service {  
    public abstract void serve();  
}  
public class ServiceImpl implements Service {  
    public void serve() { System.out.println("Service implementation is serving."); }  
}  
public class Consumer {  
    private Service service;  
    public Consumer(Service service) {  
        this.service = service;  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Service service = new ServiceImpl();  
        Consumer consumer = new Consumer(service);  
    }  
}
```


Dependency Non-Injection



```
public interface Service {  
    public abstract void serve();  
}  
public class ServiceImpl implements Service {  
    public void serve() { System.out.println("Service implementation is serving."); }  
}  
  
public class Consumer {  
    private Service service;  
    public Consumer() {  
        this.service = new Service();  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Consumer consumer = new Consumer();  
    }  
}
```

Dependency Injection



```
public interface Service {
    public abstract void serve();
}

public class ServiceImpl implements Service {
    public void serve() { System.out.println("Service implementation is serving."); }
}

public class Consumer {
    private Service service;
    public Consumer() {
        this.service = new Service();
    }
    public Consumer(Service service) {
        this.service = service;
    }
}

public class Main {
    public static void main(String[] args) {
        Service service = new ServiceImpl();
        Consumer consumer = new Consumer(service);
    }
}
```

Dependency Injection



```
public class Database {
    public abstract void connect();
}
public class RelationalDB extends Database {
    public void connect() {... }
}
public class AdvanceRelationalDB extends RelationalDB {
    public void connect() {... }
}
public class Consumer {
    private Database db;
    public Consumer(Database db) {
        this.db = db;
    }
}
public class Main {
    public static void main(String[] args) {
        Consumer consumer = new Consumer(new RelationalDB());
        Consumer consumer = new Consumer(new AdvanceRelationalDB());
    }
}
```


หัวข้อ

- Dependency Injection
- คลาสและอินเทอร์เฟซแบบ Sealed
- คลาสภายใน (Inner class หรือ Nested class)
- Model-View-Controller (MVC) Design Pattern

คลาสและอินเตอร์เฟสแบบ Sealed



```
public sealed interface InterfaceName permits Class1,Interface1, ... {  
    // Interface methods and constants  
}
```

ในภาษาจาวา sealed class คือ คลาสที่สามารถกำหนดได้ว่ามีคลาสใดบ้างที่ได้รับอนุญาตให้สืบทอดมาจากคลาสนี้ (inheritance) คุณสมบัตินี้เป็นหนึ่งในการเพิ่มความปลอดภัยของโปรแกรมและทำให้โค้ดมีความชัดเจนขึ้นในเรื่องของการสืบทอดคลาส เมื่อประกาศคลาสเป็น sealed จะต้องระบุว่ามีคลาสใดบ้างที่สามารถสืบทอดคลาสนั้นได้โดยใช้คีย์เวิร์ด permits คลาสที่สามารถสืบทอดได้นั้นต้องอยู่ในแพ็คเกจเดียวกันหรือไฟล์เดียวกันเท่านั้น คลาสที่สืบทอดมาจาก sealed class จะต้องเป็น final, sealed, หรือ non-sealed โดย

คีย์เวิร์ด	หมายความว่า
final	ไม่อนุญาตให้มีคลาสอื่นที่สามารถสืบทอดมาจากคลาสนี้ได้อีก
sealed	คลาสนี้ก็จะต้องระบุคลาสที่อนุญาตให้สืบทอดเช่นกัน
non-sealed	คลาสนี้อนุญาตให้คลาสอื่นสืบทอดได้อย่างอิสระ

คลาสและอินเทอร์เฟสแบบ Sealed



```
public sealed class Account permits SavingsAccount, FixedDepositAccount { }

public final class SavingsAccount extends Account { }           // Allowed

public final class FixedDepositAccount extends Account { }      // Allowed

public final class CheckingAccount extends Account { }          // Not Allowed
```

คลาสและอินเทอร์เฟสแบบ Sealed



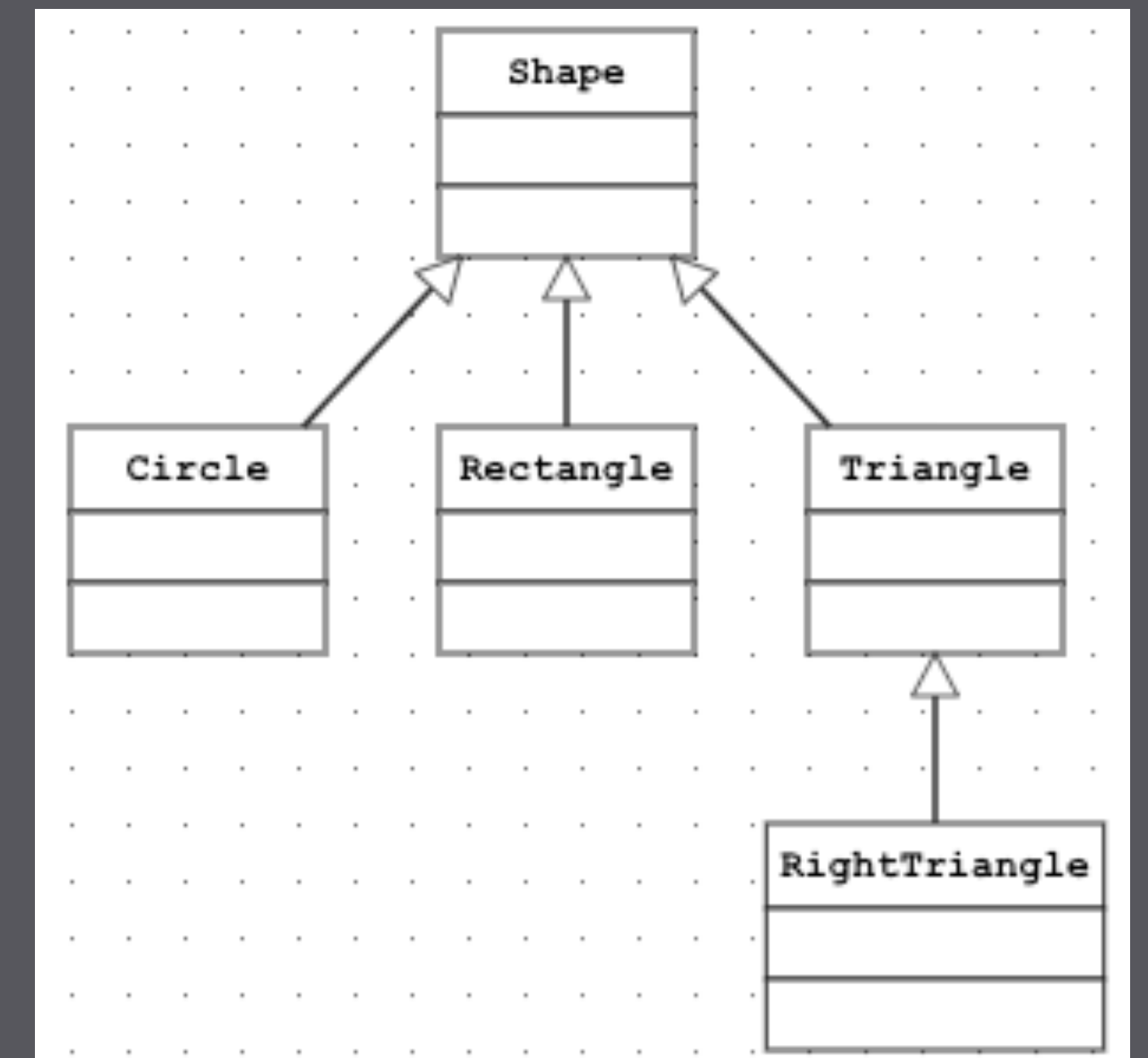
การใช้งาน sealed ในจาวามีข้อดีหลายประการ ซึ่งสามารถช่วยปรับปรุงความชัดเจนของการออกแบบโค้ดและรักษาความปลอดภัยของโปรแกรมได้

- **การควบคุมการสืบทอด (Inheritance Control):** ผู้ออกแบบคลาสสามารถจำกัดวงของการสืบทอดคลาสได้ ทำให้มั่นใจได้ว่าคลาสที่สืบทอดมาจาก sealed class จะเป็นเพียงแต่คลาสที่ได้รับการระบุไว้เท่านั้น ซึ่งช่วยป้องกันการสืบทอดที่ไม่ได้รับอนุญาต
- **การปรับปรุงการบำรุงรักษา (Maintenance Improvement):** เนื่องจากความเป็นไปได้ในการสืบทอดถูกจำกัดอยู่ เมื่อมีการเปลี่ยนแปลงใน sealed class ผู้พัฒนาจะสามารถคาดการณ์ผลกระทบที่จะเกิดขึ้นได้อย่างชัดเจน เพราะไม่มีคลาสนอกเหนือจากที่ระบุไว้ที่จะได้รับผลกระทบ
- **การรับประกันว่าไม่มีการแก้ไขหรือขยาย (Guarantee Against Modifications or Extensions):** ในบางกรณี ผู้พัฒนาอาจต้องการให้คลาสที่พวกเขาสร้างขึ้นไม่ถูกแก้ไขหรือขยายโดยคลาสอื่น นอกเหนือจากที่พวกเขาได้ระบุไว้แล้ว ซึ่ง sealed class จะให้ความสามารถนี้

คลาสและอินเทอร์เฟสแบบ Sealed



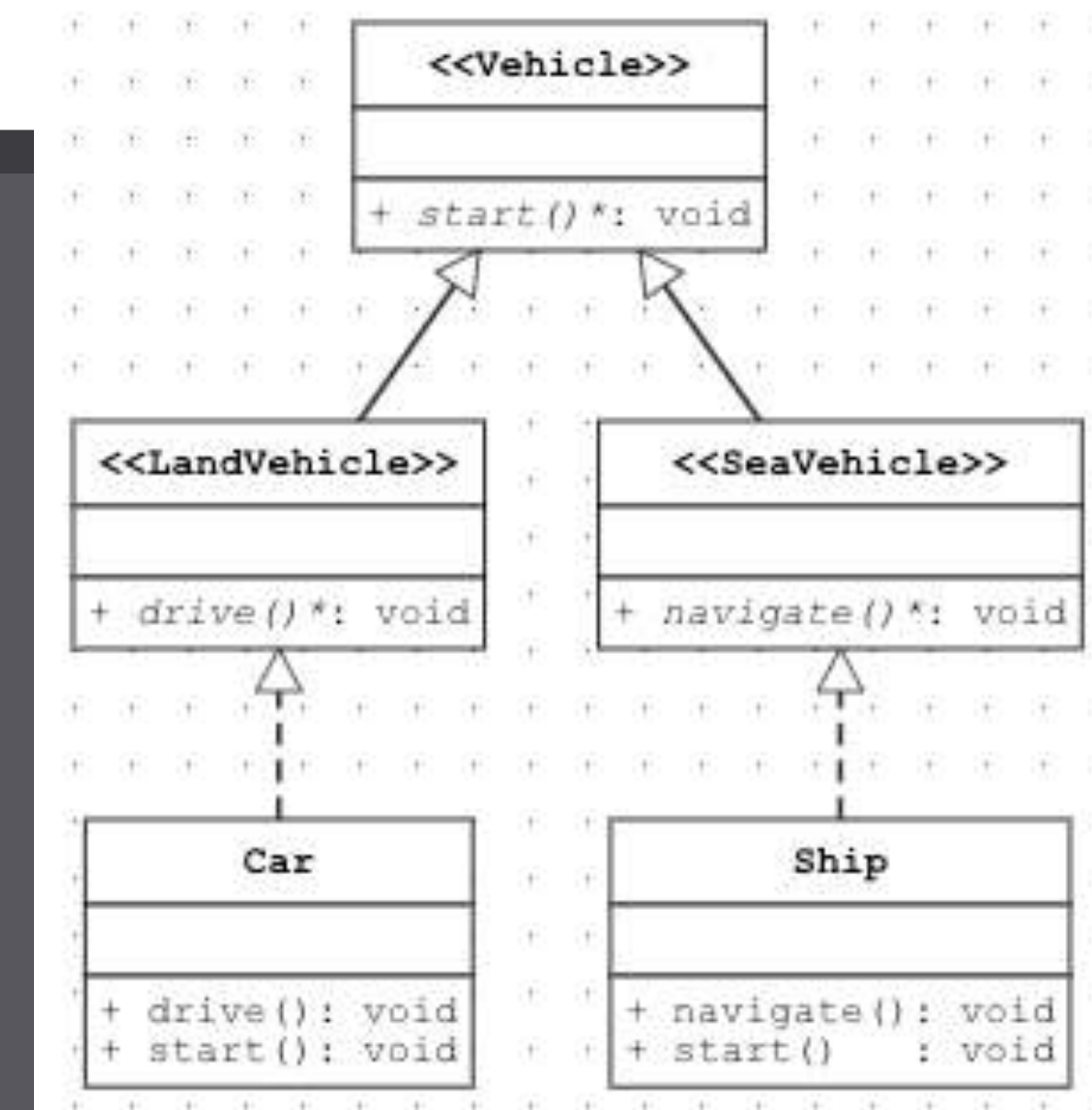
```
public sealed class Shape permits Circle, Rectangle, Triangle {  
  
}  
public final class Circle extends Shape {  
  
}  
public non-sealed class Rectangle extends Shape {  
  
}  
public sealed class Triangle extends Shape permits RightTriangle {  
  
}  
public final class RightTriangle extends Triangle {  
  
}  
  
}
```



คลาสและอินเตอร์เฟสแบบ Sealed

```

public sealed interface Vehicle permits LandVehicle, SeaVehicle {
    public abstract void start();
}
public non-sealed interface LandVehicle extends Vehicle {
    public abstract void drive();
}
public non-sealed interface SeaVehicle extends Vehicle {
    public abstract void navigate();
}
public class Car implements LandVehicle {
    public void start() { System.out.println("Car starting..."); }
    public void drive() { System.out.println("Car is driving..."); }
}
public class Ship implements SeaVehicle {
    public void start() { System.out.println("Ship starting..."); }
    public void navigate() { System.out.println("Ship is navigating..."); }
}
  
```



คลาสและอินเทอร์เฟสแบบ Sealed

```
public sealed interface Vehicle permits LandVehicle, SeaVehicle {  
    public abstract void start();  
}  
public non-sealed interface LandVehicle extends Vehicle {  
    public abstract void drive();  
}  
public sealed interface SeaVehicle extends Vehicle {  
    public abstract void navigate();  
}  
public class Car implements LandVehicle {  
    public void start() { System.out.println("Car starting..."); }  
    public void drive() { System.out.println("Car is driving..."); }  
}  
public class Ship implements SeaVehicle {  
    public void start() { System.out.println("Ship starting..."); }  
    public void navigate() { System.out.println("Ship is navigating..."); }  
}
```

error: sealed, non-sealed or final modifiers expected class Ship implements SeaVehicle {

BUILD FAILED (total time: 0 seconds)

คลาสและอินเทอร์เฟสแบบ Sealed



แนวทางการประกาศ Sealed Class โดยไม่ต้องกำหนด permits หากนักศึกษากำหนดคลาสย่อยที่ได้รับอนุญาตในไฟล์เดียวกันกับ Sealed Class ก็สามารถละเว้นส่วนคำสั่ง permits ได้ ตัวอย่างเช่น

```
// The permits clause has been omitted as its permitted classes have been defined in the same file.

package com.example.sealed.shape;
public sealed class Shape{ }

final class Circle extends Shape{
    float radius;
}
non-sealed class Square extends Shape{
    float side;
}
sealed class Rectangle extends Shape{
    float length, width;
}
final class FilledRectangle extends Rectangle {
    int red, green, blue;
}
```


หัวข้อ

- Dependency Injection
- คลาสและอินเตอร์เฟสแบบ Sealed
- คลาสภายใน (Inner class หรือ Nested class)
- Model-View-Controller (MVC) Design Pattern

คลาสภายใน



คือ คลาสที่ประกาศอยู่ภายในคลาสอื่นๆ ซึ่งบางครั้งเรียกว่าคลาสแบบซ้อน (nested class) ซึ่งคลาสภายในจะอนุญาตให้
(1) ประกาศคุณลักษณะภายในคลาสอื่นได้ และ (2) ประกาศเมธอดภายในคลาสอื่นได้

ประโยชน์ของคลาสภายใน

- ต้องการจะจัดกลุ่มของคลาสที่ต้องทำงานร่วมกัน
- ต้องการควบคุมไม่ให้มีการเข้าถึงโดยตรงจากคลาสอื่น ๆ
- ต้องการเรียกใช้คุณลักษณะหรือเมธอดของคลาสที่อยู่ภายนอกได้โดยตรง

คลาสภายในที่ใช้ในภาษาจาวาแบ่งออกเป็น 2 ประเภท

- คลาสภายในที่อยู่ภายในคลาสโดยตรง
- คลาสภายในที่อยู่ภายในเมธอด

คลาสภายในที่อยู่ภายในคลาส



การประกาศคลาสภายในคลาสอื่นที่เรียกว่า **คลาสภายนอก (Outer class)** คลาสภายในสามารถมี access modifier เป็น public, private, default หรือ protected ได้ นอกจากนี้ คลาสที่อยู่ภายในสามารถที่จะเรียกใช้คุณลักษณะและ เมธอดของคลาสภายนอกได้

การสร้างอ็อบเจกต์ของคลาสภายในมีข้อแตกต่างดังนี้

- การสร้างอ็อบเจกต์ของคลาสภายในนอกคลาสภายนอก จะต้องทำการสร้างอ็อบเจกต์ของคลาสภายนอกก่อน แล้วจึงสร้างอ็อบเจกต์ของคลาสภายในได้
- การสร้างอ็อบเจกต์ภายในเมธอดที่อยู่ในคลาสภายนอกสามารถทำได้โดยตรง

คลาสภายในที่อยู่ภายในคลาส

```
public class Outer {  
    public void method1() {  
        new Inner().method3(); // การสร้างอ็อบเจกต์ภายในเมธอดที่อยู่ในคลาสภายนอกสามารถทำได้โดยตรง  
        System.out.println("Hello Method 1");  
    }  
    public void method2() { System.out.println("Hello Method 2"); }  
    public class Inner {  
        public void method3() { System.out.println("Hello Method 3"); }  
        public void method4() {  
            method2();  
            System.out.println("Hello Method 4");  
        }  
    }  
}  
public static void main(String args[]) {  
    Outer out = new Outer();  
    Outer.Inner inner = new Outer().new Inner();  
    out.method1();  
    out.method2();  
    inner.method3();  
    inner.method4();  
}}
```

การสร้างอ็อบเจกต์ของคลาสภายในนอกคลาส
ภายนอก จะต้องทำการสร้างอ็อบเจกต์ของคลาส
ภายนอกก่อน แล้วจึงสร้างอ็อบเจกต์ของคลาส
ภายในได้

```
run:  
Hello Method 3  
Hello Method 1  
Hello Method 2  
Hello Method 3  
Hello Method 2  
Hello Method 4
```


คลาสภายในที่อยู่ภายในคลาส

```
public class Outer {  
    public void method1() {  
        new Inner().method3();  
        System.out.println("Hello Method 1");  
    }  
    public void method2() { System.out.println("Hello Method 2"); }  
    public class Inner {  
        public void method3() { System.out.println("Hello Method 3"); }  
        public void method4() {  
            method2();  
            System.out.println("Hello Method 4");  
        }  
    }  
    public static void main(String args[]) {  
        Outer out = new Outer();  
        (out.new Inner()).method3();  
        //out.new Inner().method3();  
    }  
}
```

output

Hello Method 3

คลาสภายในที่อยู่ภายในคลาส

```
public class Outer {  
    public void method1() {  
        new Inner().method3();  
        System.out.println("Hello Method 1");  
    }  
    public void method2() { System.out.println("Hello Method 2"); }  
    public class Inner {  
        public void method3() { System.out.println("Hello Method 3"); }  
        public void method4() {  
            method2();  
            System.out.println("Hello Method 4");  
        }  
    }  
    public static void main(String args[]) {  
        Outer out = new Outer();  
        (out.new Inner()).method1();  
    }  
}
```

```
Exception in thread "main" java.lang.RuntimeException: Uncompilable code - cannot find symbol  
symbol:   method method1()  
location: class Outer.Inner  
|         at Outer.main(Outer.java:1)
```

คลาสภายในที่อยู่ภายในคลาส

```
public class Outer {  
    public void method1() {  
        new Inner().method3();  
        System.out.println("Hello Method 1");  
    }  
    public void method2() { System.out.println("Hello Method 2"); }  
    public class Inner {  
        public void method3() { System.out.println("Hello Method 3"); }  
        public void method4() {  
            method2();  
            System.out.println("Hello Method 4");  
        }  
    }  
}  
public static void main(String args[]) {  
    Outer.Inner inner = new Outer().new Inner();  
    inner.method1();  
}  
}
```

```
Exception in thread "main" java.lang.RuntimeException: Uncompilable code - cannot find symbol  
symbol:   method method1()  
location: variable inner of type Outer.Inner  
at Outer.main(Outer.java:1)
```


คุณสมบัติที่สำคัญอื่น ๆ ของคลาสภายใน

- คลาสภายในอาจเป็นคลาสแบบ **abstract** หรือ **อินเทอร์เฟส** ได้
- **คลาสภายในที่อยู่ภายในคลาสนอกโดยตรง** ถ้ามี **modifier** เป็น **static** จะกลายเป็นคลาสปกติ และสามารถสร้างอ็อบเจกต์โดยการเรียกชื่อของคลาสนอกได้ดังนี้

```
Outer.Inner in3 = Outer.new Inner();
```

- คลาสภายในไม่สามารถที่ประกาศให้มีคุณลักษณะหรือเมธอดเป็นแบบ `static` ได้ เว้นแต่คลาสภายในจะเป็นคลาสแบบ `static`
- **คลาสภายใน**สามารถที่จะใช้ตัวแปรที่เป็นคุณลักษณะของคลาสหรือคุณลักษณะของอ็อบเจกต์ของคลาสนอกได้

คลาสภายในที่อยู่ภายในเมธอด



คลาสภายในประเภทนี้จะมี `access modifier` เป็น **default** เท่านั้นและจะ**ไม่สามารถที่จะสร้าง**อ็อบเจกต์ของคลาสภายในประเภทนี้**นอกเมธอดที่ประกาศคลาส**ได้ แต่จะสามารถกำหนด `final` และ `abstract` ได้ คลาสประเภทนี้จะสามารถเรียกใช้

- **ตัวแปรภายในของเมธอด**ได้ในกรณีที่ตัวแปรนั้นประกาศเป็น **final** เท่านั้น
- **ตัวแปรที่เป็นคุณลักษณะ**ของคลาสหรือคุณลักษณะของอ็อบเจกต์ สามารถเรียกใช้ได้เช่นเดียวกับคลาสภายในที่อยู่ภายในคลาส

ตัวอย่างโปรแกรมแสดงคลาสที่อยู่ภายในเมธอด

```
public class MOuter {  
    private int a = 1;  
    public void method(final int b) {  
        final int c = 2;  
        int d = 3;  
        class Inner {  
            private void iMethod() {  
                System.out.println("a = "+a);  
                System.out.println("b = "+b);  
                System.out.println("c = "+c);  
                //System.out.println("d = "+d);    //illegal  
            }  
        }  
        Inner y = new Inner();  
        y.iMethod();  
    }  
}  
  
public class Main {  
    public static void main(String args[]) {  
        MOuter m = new MOuter();  
        m.method(5);  
    }  
}
```

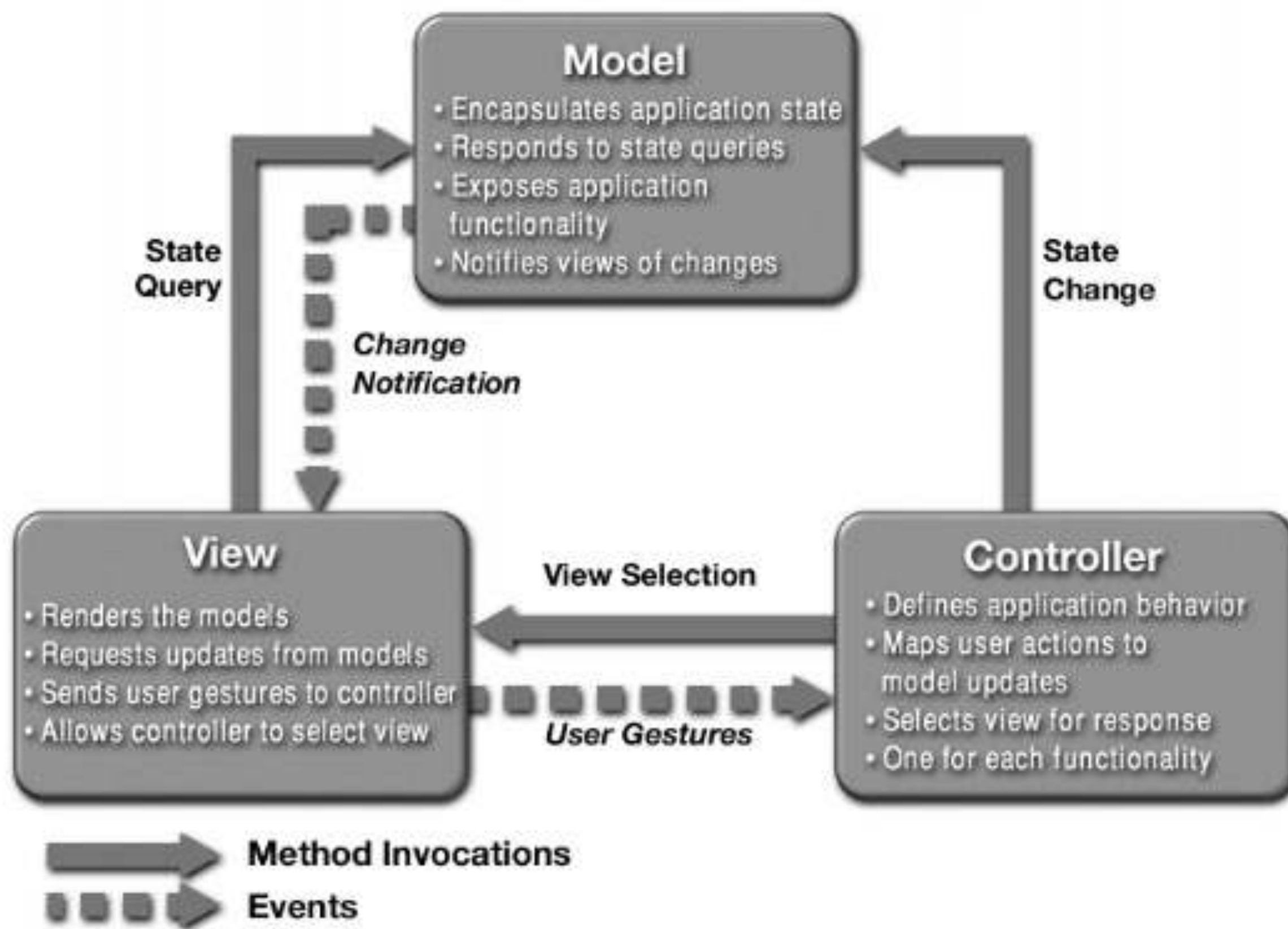
ผลลัพธ์

a = 1
b = 5
c = 2

หัวข้อ

- Dependency Injection
- คลาสและอินเตอร์เฟสแบบ Sealed
- คลาสภายใน (Inner class หรือ Nested class)
- Model-View-Controller (MVC) Design Pattern

หลักการทำงานของ MVC



MVC

คือ Design Pattern รูปแบบหนึ่งที่มีการแบ่งแยกระบบออกเป็น 3 ส่วนหลัก ได้แก่ Model, View และ Controller ซึ่งในยุคเริ่มแรกมีความพยายามที่จะแยกส่วนของ Model และ View ออกจากกัน

Controller

คือ คลาสที่จัดการกับคลาส Model โดยขึ้นอยู่กับการทำงานที่ได้จากคลาส View และยังทำหน้าที่สรรหาข้อมูลจากคลาส Model เพื่อนำไปแสดงผลที่คลาส View

Model

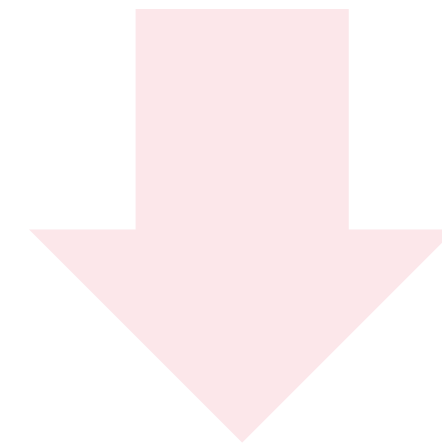
คือ คลาสที่เกี่ยวข้องกับชนิดข้อมูล คลาสของฟังก์ชันพื้นฐาน และคลาสที่เชื่อมต่อกับฐานข้อมูลสำหรับการพัฒนาโปรแกรมเชิงวัตถุ

View

คือ คลาสที่ใช้เพื่อแสดงผลลัพธ์ผ่านทาง GUI ซึ่งมีการทำงานสัมพันธ์อยู่กับคลาส controller

ทำไมต้องใช้ MVC

การออกแบบโปรแกรมที่ไม่รอบคอบและขาดประสิทธิภาพของผู้พัฒนาโปรแกรม เมื่อพัฒนาไปเรื่อย ๆ โปรแกรมจะมีขนาดใหญ่ขึ้นเรื่อย ๆ สิ่งนี้อาจจะส่งผลให้ไฟล์มีปริมาณที่มากยิ่งขึ้นแล้ว จัดลำดับไม่เรียบร้อย โปรแกรมก็ยาว อ่านก็ยาก และไม่สะดวกต่อการทำงานเป็นทีม

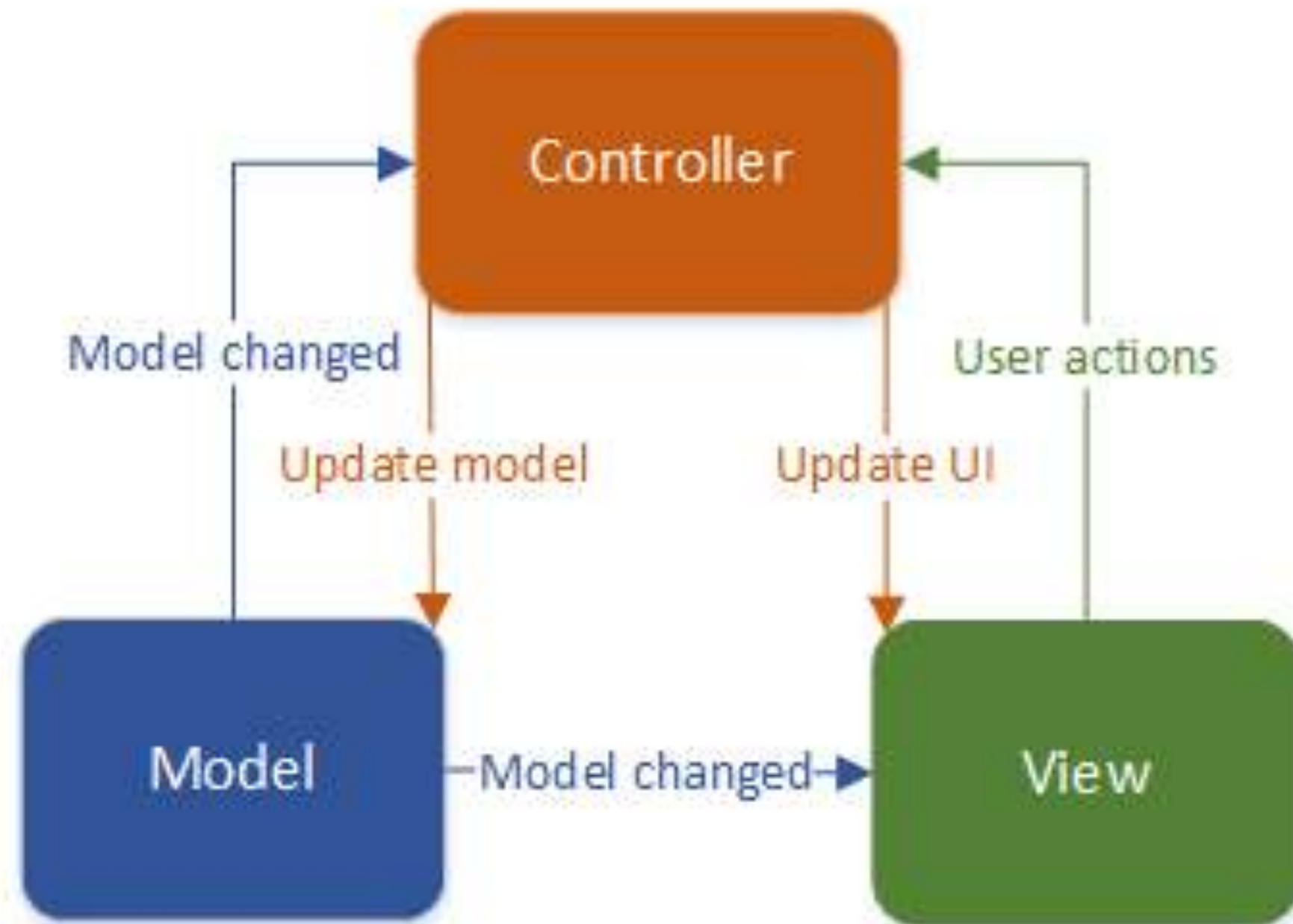


ดังนั้น การออกแบบโปรแกรมในรูปแบบ MVC จะทำให้สามารถทำงานได้ง่ายและสะดวกยิ่งขึ้น

ข้อควรระวัง

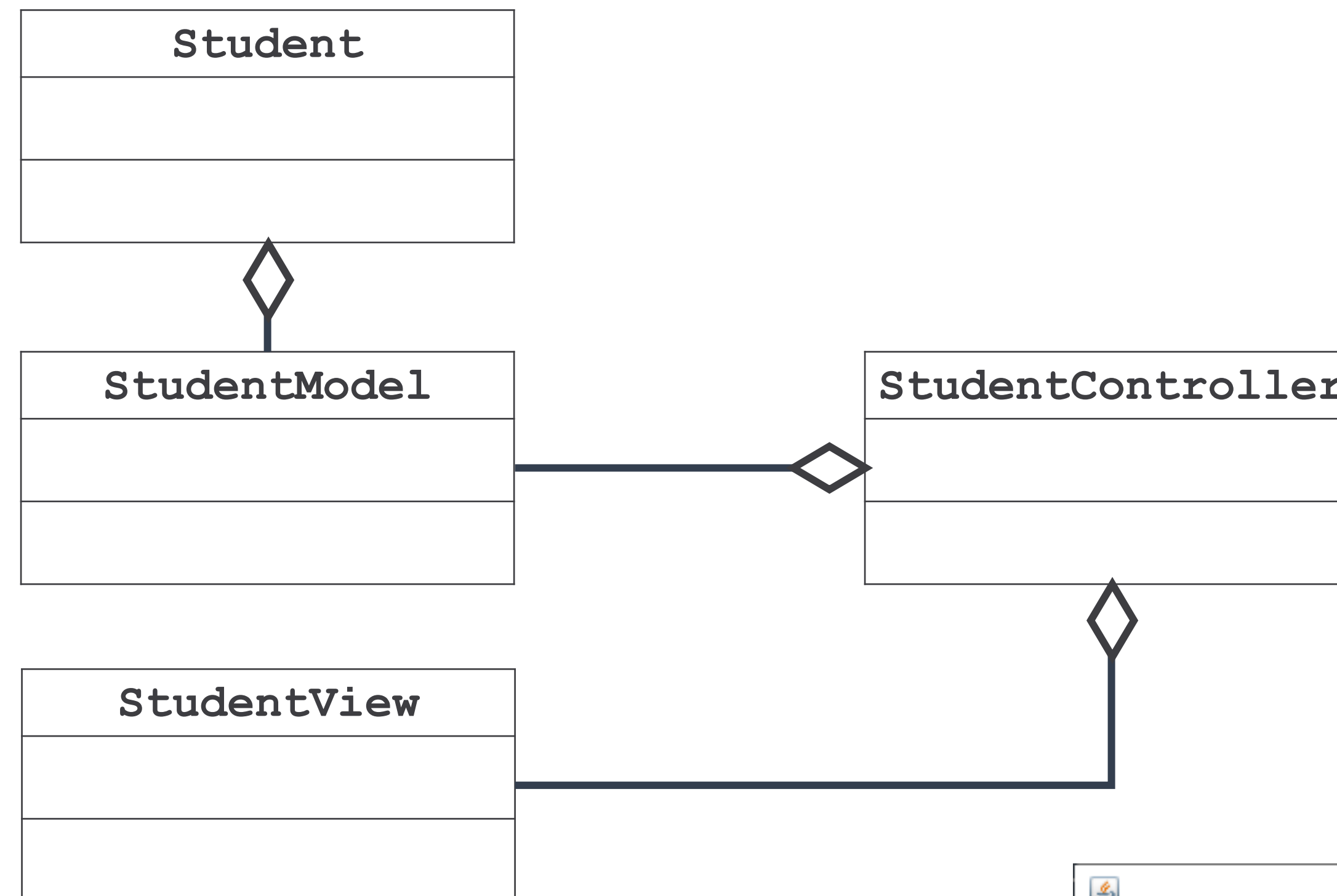
หลักการสำคัญใน MVC คือ “แยกส่วนของ Model และ View ออกจากกัน”

MVC



1. คลาสประเภท Model ไม่สามารถเรียก Method จากคลาสประเภท View หรือ Controller ได้โดยตรง เนื่องจากคลาสประเภท Model จะไม่มีตัวแปรที่เก็บ Object ของคลาสประเภท View และ Controller ไว้ แต่อาศัยหลักการ Observer โดยใช้ Object พวก Observer ที่ Implements มาจาก Observer Interface จากนั้นคลาสประเภท Model จะแจ้งข้อมูลโดยการส่ง Event Notification กลับไปหา Observer ตัวอื่น ๆ ถ้า Observer นั้นเป็น View มันก็จะรับข้อมูลมา Update ข้อมูลในตัวเองใหม่
2. คลาสประเภท View ไม่สามารถเรียก Method จากคลาส Controller ได้ แต่อาศัยการส่ง Event Notification ไปหาคลาสประเภท Observer ของ Controller ตัวนั้น ๆ เท่านั้น

สรุปความเชื่อมโยงของตัวอย่างที่ 1

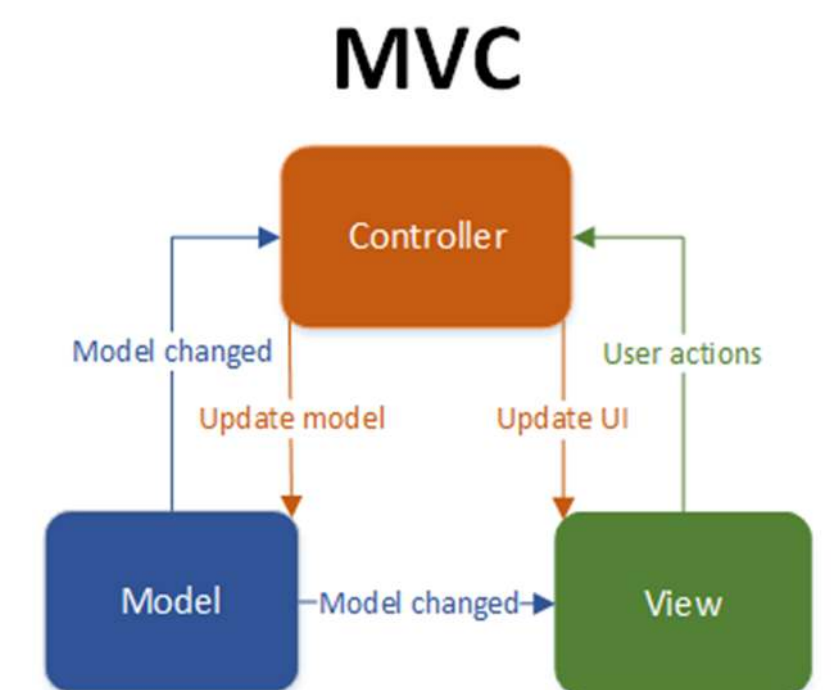
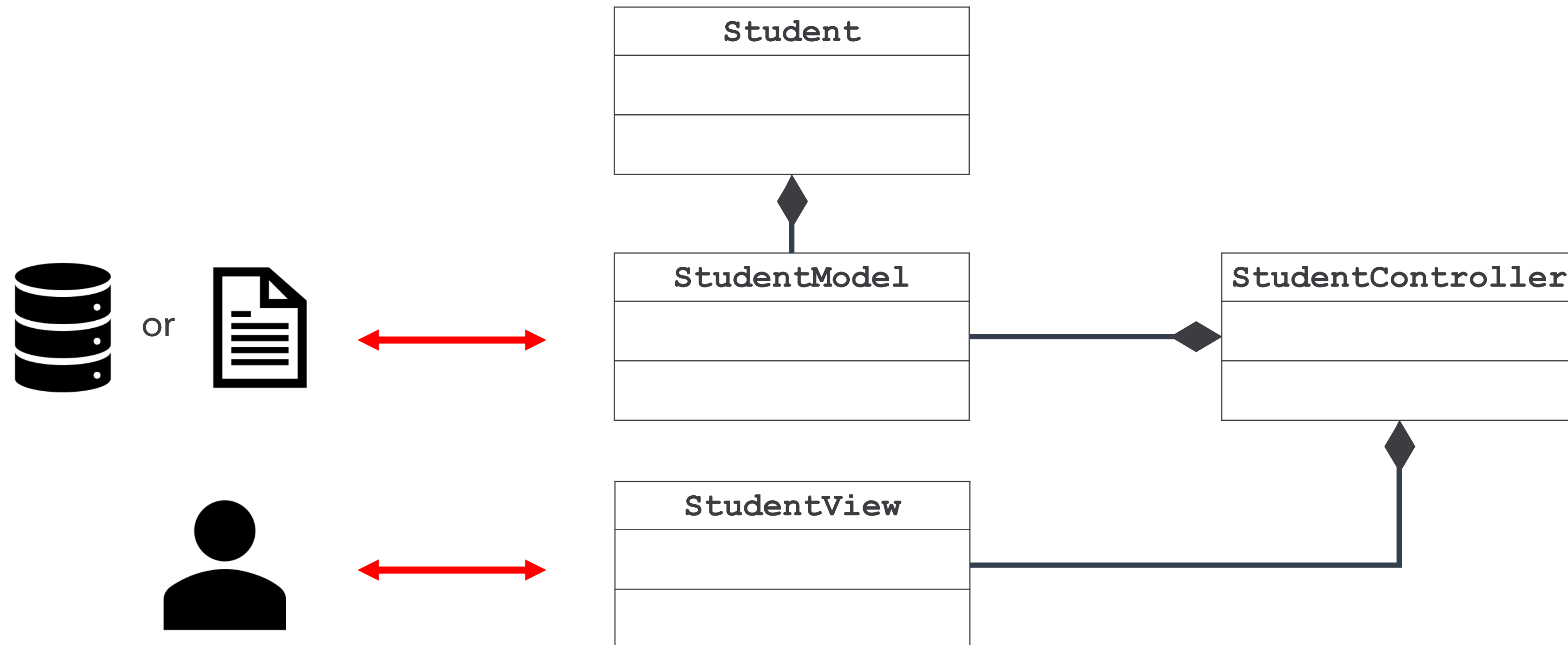


```

public class Main{
    public static void main(String[] args) {
        new StudentController();
    }
}
  
```

Student ID
62070011
Student Name
bank
Student GPA
3.6
Update Complete !!

สรุปความเชื่อมโยงของตัวอย่างที่ 1



ตัวอย่างที่ 1: Student

```
import java.io.Serializable;
public class Student implements Serializable {
    private String ID;
    private String name;
    private double GPA;

    public Student(String ID, String name, double GPA) {
        this.ID = ID;
        this.name = name;
        this.GPA = GPA;
    }

    public String getID() { return ID; }
    public void setID(String ID) { this.ID = ID; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getGPA() { return GPA; }
    public void setGPA(double GPA) { this.GPA = GPA; }
}
```

ตัวอย่างที่ 1: StudentModel

```
import java.io.*;
public class StudentModel {
    private Student data;
    public StudentModel() {
        data = new Student("", "", 0);
    }
    public boolean loadData() {
        File f = new File("dataStudent.dat");
        if (f.exists()) {
            try (FileInputStream fin = new FileInputStream("dataStudent.dat");
                ObjectInputStream in = new ObjectInputStream(fin);) {
                data = (Student) in.readObject();
                return true;
            } catch (Exception i) {
                return false;
            }
        }
        return false;
    }
}
// ต่อหน้าถัดไป
```

ตัวอย่างที่ 1: StudentModel

```
public boolean saveData() {  
    try (FileOutputStream fOut = new FileOutputStream("dataStudent.dat");  
         ObjectOutputStream oout = new ObjectOutputStream(fOut);) {  
        oout.writeObject(data);  
        return true;  
    } catch (Exception i) {  
        return false;  
    }  
  
}  
  
public Student getData() {  
    return data;  
}  
  
public void setData(Student data) {  
    this.data = data;  
}  
  
}
```


ตัวอย่างที่ 1: StudentView

```
import java.awt.*;
import javax.swing.*;

public class StudentView {
    private JFrame fr;
    private JLabel lbl1, lbl2, lbl3, lbl4;
    private JButton btn1;
    private JTextField txt1, txt2, txt3;

    public StudentView() {
        fr = new JFrame();
        lbl1 = new JLabel("Student ID");
        lbl2 = new JLabel("Student Name");
        lbl3 = new JLabel("Student GPA");
        lbl4 = new JLabel("");

        btn1 = new JButton("Update");

        txt1 = new JTextField("", 50);
        txt2 = new JTextField("", 50);
        txt3 = new JTextField("", 50);
```

```
        fr.setLayout(new FlowLayout());
        fr.add(lbl1);          fr.add(txt1);
        fr.add(lbl2);          fr.add(txt2);
        fr.add(lbl3);          fr.add(txt3);
        fr.add(btn1);          fr.add(lbl4);

        fr.setSize(600, 250);
        fr.setVisible(true);
        fr.setDefaultCloseOperation
            (JFrame.EXIT_ON_CLOSE);
    }

    public void setData(String i,
                        String n,
                        String g) {

        dataStudent = s;
        txt1.setText(i);
        txt2.setText(n);
        txt3.setText(g);
    }
    // setter() and getter()
}
```

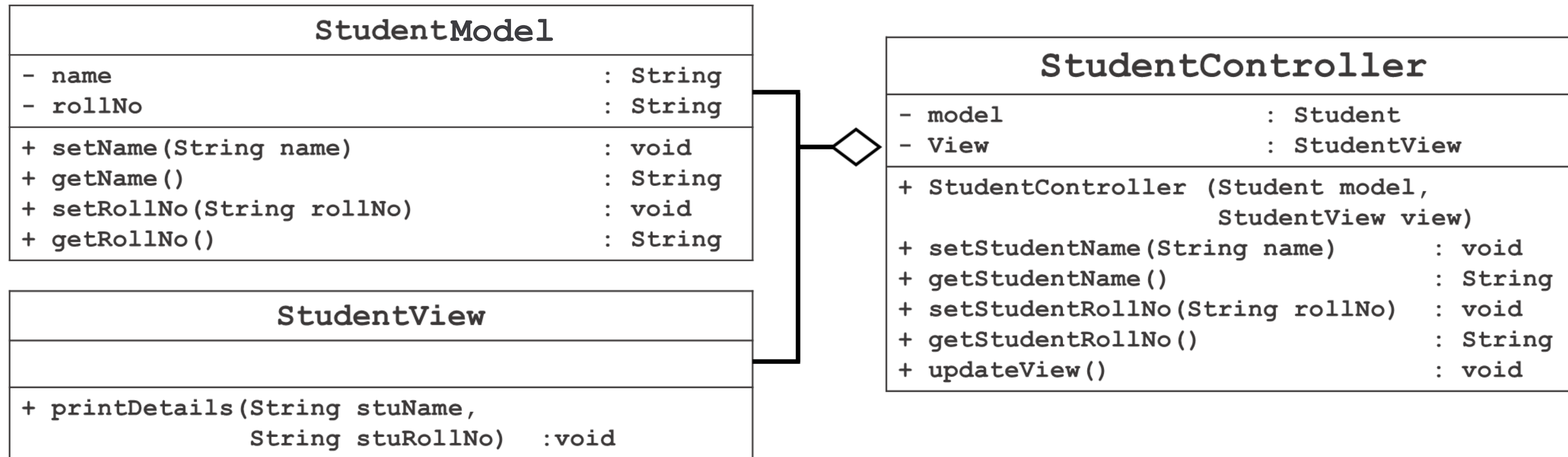

ตัวอย่างที่ 1: StudentController

```
public class StudentController implements ActionListener {
    private StudentView view;
    private StudentModel model;
    public StudentController(){
        view = new StudentView();
        model = new StudentModel();
        init();
    }
    public void init(){
        if(model.loadData()){
            view.setData(
                model.getData().getID(),
                model.getData().getName(),
                model.getData().getGPA()+""
            );
        }
        view.getBtn1().addActionListener(this);
    }
    // ActionPerform() Code
}
```

ตัวอย่างที่ 1: StudentController

```
public void actionPerformed(ActionEvent ae) {  
  
    if (ae.getSource().equals(view.getBtn1())) {  
  
        model.getData().setID(view.getTxt1().getText());  
        model.getData().setName(view.getTxt2().getText());  
        model.getData().setGPA(Double.parseDouble(view.getTxt3().getText()));  
  
        if (model.saveData())  
            view.getLbl4().setText("Complete !!");  
        else  
            view.getLbl4().setText("Fail !!");  
    }  
}
```

ตัวอย่างที่ 2: การใช้งาน MVC ร่วมกัน



```

public class Main {
    public static void main(String[] args) {
        StudentController sc = new StudentController();
        sc.setStudentName("Robert");
        sc.setStudentRollNo("10");
        sc.updateView();
    }
}
  
```

Output - MVCDemo2 (run) ×

```

run:
Student:
Name: null
Roll No: null
Student:
Name: Robert
Roll No: 10
BUILD SUCCESSFUL (total time: 0 seconds)
  
```


ตัวอย่างที่ 2: StudentModel

```
public class StudentModel {  
    private String rollNo;  
    private String name;  
    public String getRollNo() {  
        return rollNo;  
    }  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

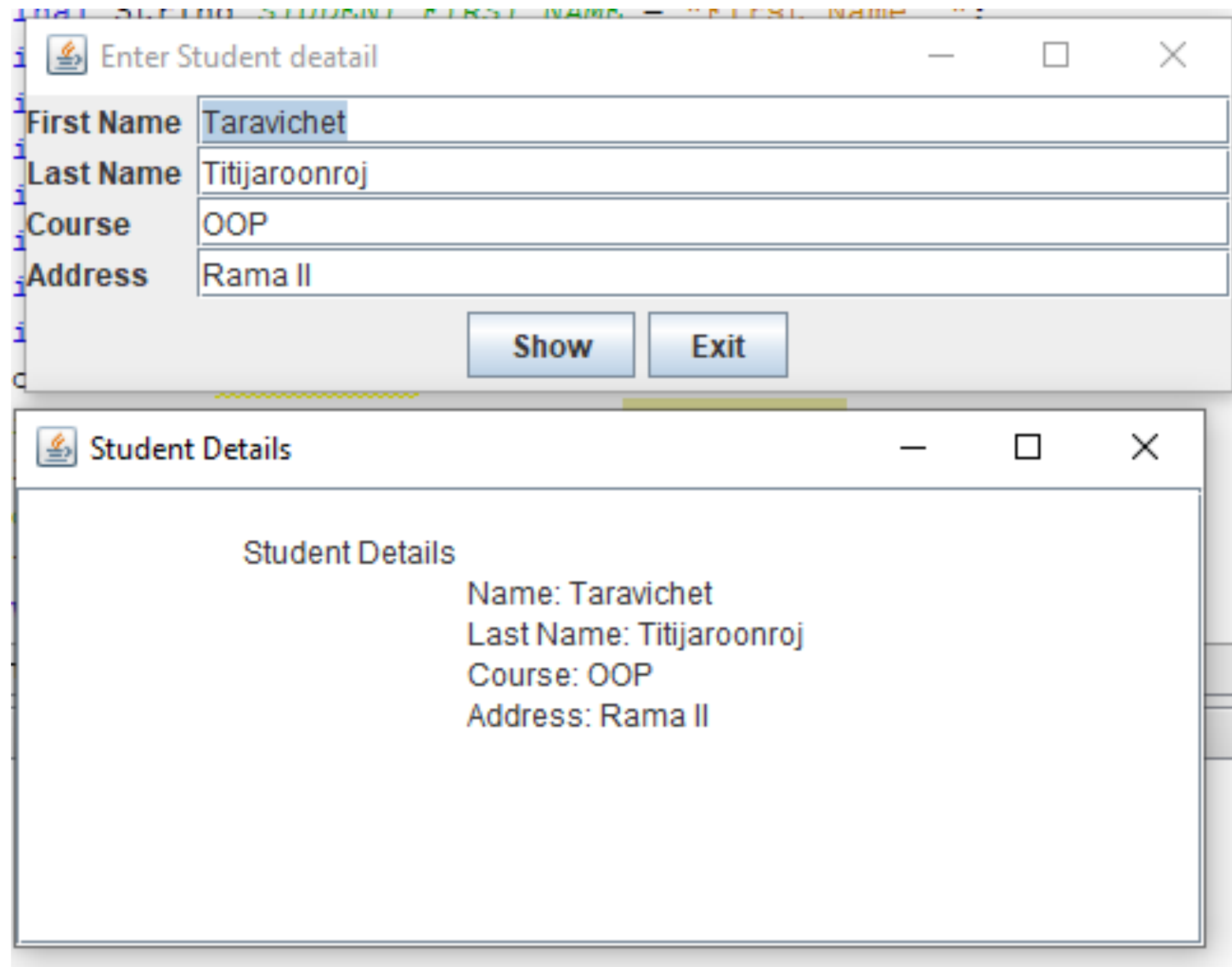
ตัวอย่างที่ 2: StudentView

```
public class StudentView {  
    public void printDetails(String studentName, String studentRollNo) {  
        System.out.println("Student: ");  
        System.out.println("Name: " + studentName);  
        System.out.println("Roll No: " + studentRollNo);  
    }  
}
```


ตัวอย่างที่ 2: StudentController

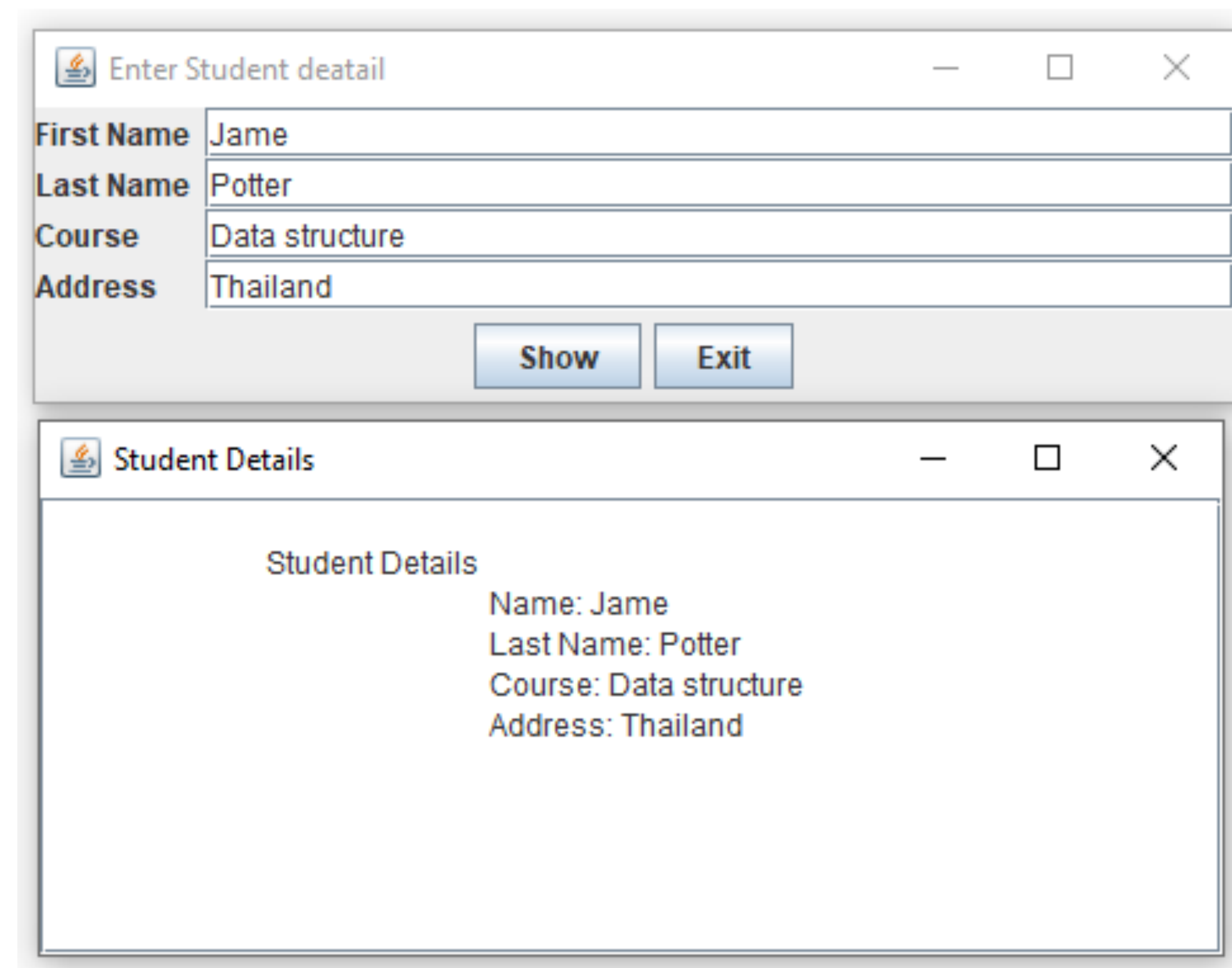
```
public class StudentController {
    private StudentModel model;
    private StudentView view;
    public StudentController() {
        model = new StudentModel();
        view = new StudentView();
        this.updateView();
    }
    public void setStudentName(String name) {
        model.setName(name);
    }
    public String getStudentName() {
        return model.getName();
    }
    public void setStudentRollNo(String rollNo) {
        model.setRollNo(rollNo);
    }
    public String getStudentRollNo() {
        return model.getRollNo();
    }
    public void updateView() {
        view.printDetails(model.getName(), model.getRollNo());
    }
}
```

ตัวอย่างที่ 3



The screenshot shows two windows. The top window, titled "Enter Student deatail", contains four text input fields: "First Name" (containing "Taravichet"), "Last Name" (containing "Titijaroonroj"), "Course" (containing "OOP"), and "Address" (containing "Rama II"). Below the fields are two buttons: "Show" and "Exit". The bottom window, titled "Student Details", displays the entered information in a text area:

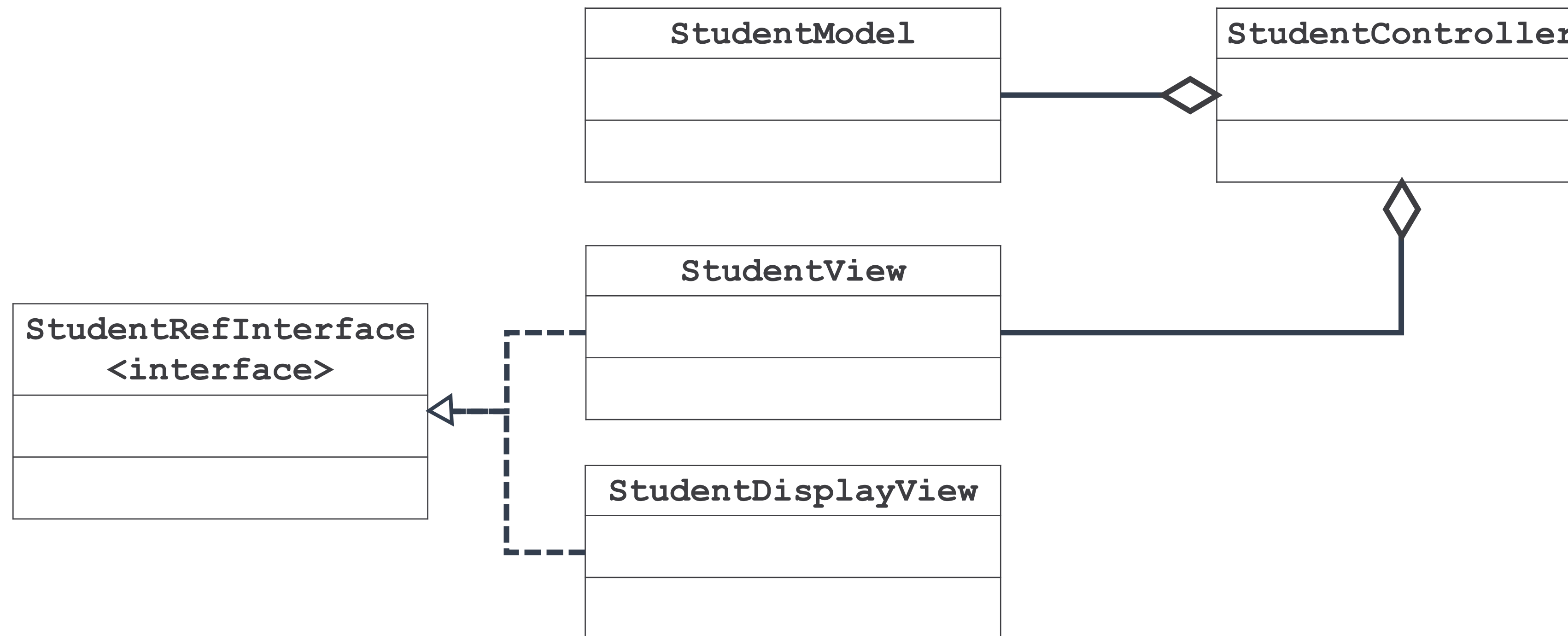
```
Student Details
Name: Taravichet
Last Name: Titijaroonroj
Course: OOP
Address: Rama II
```



The screenshot shows two windows. The top window, titled "Enter Student deatail", contains four text input fields: "First Name" (containing "Jame"), "Last Name" (containing "Potter"), "Course" (containing "Data structure"), and "Address" (containing "Thailand"). Below the fields are two buttons: "Show" and "Exit". The bottom window, titled "Student Details", displays the entered information in a text area:

```
Student Details
Name: Jame
Last Name: Potter
Course: Data structure
Address: Thailand
```

สรุปรูปความเชื่อมโยงของตัวอย่างที่ 3



```

public class Main {
    public static void main(String[] arguments) {
        new StudentController();
    }
}
  
```


ตัวอย่างที่ 3: StudentModel

```
public class StudentModel {
    private String firstName, lastName, course, address;
    private ArrayList studentViews = new ArrayList();
    public StudentModel() { this(null); }
    public StudentModel(StudentRefInterface view) {
        firstName = "";    lastName = "";    course = "";    address = "";
        if (view != null)
            studentViews.add(view);
    }
    public void addContactView(StudentRefInterface view) {
        if (!studentViews.contains(view))
            studentViews.add(view);
    }
    public void removeContactView(StudentRefInterface view) { studentViews.remove(view); }

    public void updateModel(String newFName, String newLName, String newTitle, String newOrg) {
        setFirstName(newFName);    setLastName(newLName);    setCourse(newTitle);    setAddress(newOrg);
        updateView();
    }
    private void updateView() {
        Iterator notifyViews = studentViews.iterator();
        while (notifyViews.hasNext())
            ((StudentRefInterface) notifyViews.next()).refresh(firstName, lastName, course, address);
    }
    // setter() และ getter()
}
```

ตัวอย่างที่ 3:StudentRefInterface และ StudentDisplayView

```
public interface StudentRefInterface {
    public void refresh(String first, String last, String course,String address);
}

public class StudentDisplayView implements StudentRefInterface {
    private JTextArea studentDetail;
    private JPanel p;
    private JFrame fr;
    private JScrollPane scrollDisplay;
    public StudentDisplayView() {
        fr = new JFrame();          p = new JPanel();
        p.setLayout(new BorderLayout());
        studentDetail = new JTextArea(10, 40);
        studentDetail.setEditable(false);
        scrollDisplay = new JScrollPane(studentDetail);
        p.add(scrollDisplay, BorderLayout.CENTER);
        fr.add(p);          fr.pack();          fr.setVisible(true);
    } public void refresh(String newFirstName, String newLastName, String newCourse, String newAddress) {
        studentDetail.setText("\n\tStudent Details\n\t" + "\tName: "
            + newFirstName + "\n\t\tLast Name: " + newLastName + "\n"
            + "\t\tCourse: " + newCourse + "\n" + "\t\tAddress: "
            + newAddress);
    }
}
```


ตัวอย่างที่ 3: StudentView

```
public class StudentView implements StudentRefInterface {
private JLabel firstNameLabel, lastNameLabel, courseLabel, addressLabel;
private JTextField firstName, lastName, course, address;
private JButton display, exit;
private JPanel labelPanel, controlPanel, p ;
private JFrame fr;
public StudentView() {
    fr = new JFrame();          p = new JPanel();      display = new JButton("Show");
    exit = new JButton("Exit");  firstNameLabel = new JLabel("First Name ");
    lastNameLabel = new JLabel("Last Name ");          courseLabel = new JLabel("Course ");
    addressLabel = new JLabel("Address ");             firstName = new JTextField();
    lastName = new JTextField();      course = new JTextField();      address = new JTextField();

    labelPanel = new JPanel();
    labelPanel.setLayout(new GridLayout(4, 2));

    labelPanel.add(firstNameLabel);      labelPanel.add(firstName);      labelPanel.add(lastNameLabel);
    labelPanel.add(lastName);            labelPanel.add(courseLabel);      labelPanel.add(course);
    labelPanel.add(addressLabel);        labelPanel.add(address);
    controlPanel = new JPanel();
    controlPanel.add(display);            controlPanel.add(exit);

    p.setLayout(new BorderLayout());
    p.add(labelPanel);                    p.add(controlPanel, BorderLayout.SOUTH);
    fr.add(p);                            fr.pack();                            fr.setVisible(true);
}
```

ตัวอย่างที่ 3: StudentView (ต่อ)



```
//ต่อ StudentView
public Object getUpdateRef() {      return display; }
public String getFirstName() {      return firstName.getText(); }
public String getLastName() {      return lastName.getText(); }
public String getCourse() {         return course.getText(); }
public String getAddress() {        return address.getText(); }
public JButton getDisplay() {        return display; }
public void setDisplay(JButton display) { this.display = display; }
public JButton getExit() { return exit; }
public void setExit(JButton exit) { this.exit = exit; }

public void refresh(String newFirstName, String newLastName, String newTitle, String newOrganization) {
    firstName.setText(newFirstName);
    lastName.setText(newLastName);
    course.setText(newTitle);
    address.setText(newOrganization);
}

}
```

ตัวอย่างที่ 3: StudentController



```
public class StudentController implements ActionListener {
    private StudentModel model;
    private StudentView view;
    private StudentDisplayView displayView;
    public StudentController() {
        model = new StudentModel();
        view = new StudentView();
        displayView = new StudentDisplayView();

        model.addContactView(displayView);
        model.addContactView(view);

        view.getDisplay().addActionListener(this);
        view.getExit().addActionListener(this);
    } public void actionPerformed(ActionEvent evt) {
        Object source = evt.getSource();
        if (evt.getSource().equals(view.getDisplay()))
            updateModel();
        else if (evt.getSource().equals(view.getExit()))
            System.exit(0);

    }
    private void updateModel() {
        model.updateModel(view.getFirstName(), view.getLastName(), view.getCourse(), view.getAddress());
    }
}
```